Intro To Operating Systems

2026 Semester 1 SOFTENG 370: Operating Systems

Lecture 1

Talia Xu

2.0.1

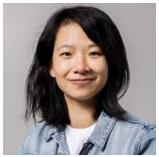
About Me

Lecturer in Computer Science

Research Interest: Embedded Systems, Low Power Wireless Communication, IoT /Batteryless Systems
I mostly teach operating systems, computer organization

Agenda

Your Instructors



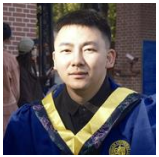
Talia Xu

Instructor: Week 1 - 6
Office: 303S-592
Email: talia.xu@auckland.ac.nz

Sean Ma
(Course Director/Coordinator)

Instructor: Week 7 - 12
Office: 303S-587
Email: sean.ma@auckland.ac.nz

Your Tutors



Zhihang Liu

Tutorial: Thurs 4pm - 5pm (Room 405-460)
Office: 303S-G87
Email: zliu604@aucklanduni.ac.nz

Ed Discussion

Evaluation for this course

Assessment	Weight	Due Date
Lab 1	7.5%	March 23
Midterm	20%	April 11 @ 6:30 pm (tentative)
Lab 2	7.5%	April 20
Lab 3	7.5%	May 9 (tentative)
Final	50%	TBD

Double Pass Requirement: Must separately pass both practical and theory components.

Double Pass Requirement

Must separately pass both practical and theory components.

Aegrotat / Compassionate Consideration Policy

If circumstances impacted your ability to sit an invigilated assessment, or impaired your preparation/performance, we will organise an alternative assessment. The alternative assessment may be in the same format or in an alternative format such as viva voce/oral test.

The Recommended Books Complement Lectures

"Operating Systems: Three Easy Pieces"
by Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau

"Silberschatz's Operating System Concepts, 10th Edition, Global Edition"
by Abraham Silberschatz and Peter B. Galvin and Greg Gagne

"The C Programming Language"
by Brian Kernighan and Dennis Ritchie

Skills You Should Practice Again If Needed

Programming

- C Programming and Debugging
- Python Programming and Debugging

Basic Concepts

- Being able to convert between binary, hex, and decimal
- Little-endian and big-endian
- Memory being byte-addressable, memory addresses (pointers)

Why Operating Systems?

Understanding the operating system will make you a better programmer

You will either write software that:

- Interacts with the operating system
- Is the operating system

Not always important if you only care about functionality.

Important when performance matters in your program.

Why Operating Systems?

```
int array[ROWS][COLS];
for (int i = 0; i < ROWS; i++) {
    for (int j = 0; j < COLS; j++) {
        array[i][j] = i * COLS + j;
    }
}
```

```
int array[ROWS][COLS];
for (int j = 0; j < COLS; j++) {
    for (int i = 0; i < ROWS; i++) {
        array[i][j] = i * COLS + j;
    }
}
```

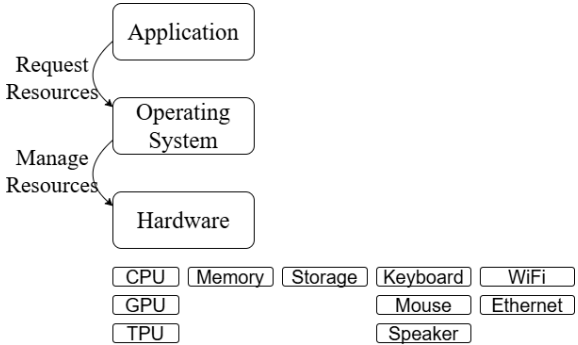
Why Operating Systems?

```
// 1 + 2 + 3 + 4 + 5 + ... + n = 100

int  iterative_addition(int  n) {
    int  sum = 0;
    for (int  i = 0; i < n; i++) {
        sum += i;
    }
    return sum;
}

int  recursive_addition(int  n) {
    if (n == 0) return 0;
    return (n - 1) + recursive_addition(n - 1);
}
```

An Operating System Manages Resources



Three Core Operating System Concepts

Virtualization lets each program think it has the whole computer to itself

Concurrency allows multiple tasks to appear to happen at once

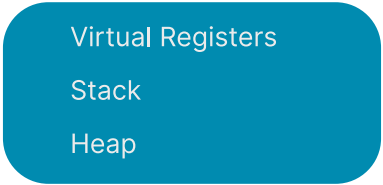
Persistence guarantees that data will stick around and can be retrieved later

Our First Abstraction is a Process

Program: a file containing all the instructions and data required to run

Process: an instance of running a program

The Basic Requirements for a Process



What Happens If Two Processes Run the Same Program?

```c
#include <stdio.h>
#include <unistd.h>

static int global = 0;

int main(void) {
  int local = 0;
  while (1) {
    ++local;
    ++global;
    printf("local = %d, global = %d\n", local, global);
    printf("0x%lx, 0x%lx\n", (long int)&local, (long int)&global);
    sleep(1);
  }
  return 0;
}
```

The Basic Requirements for a Process

How are you able to run two different programs at the same time?

For example, a “hello world” program and another that counts up one every second

Does the OS Allocate Different Stacks For Each Process?

The stacks for each process need to be in physical memory

One option is the operating system just allocates any unused memory for the stack

Would there be any issues with this?

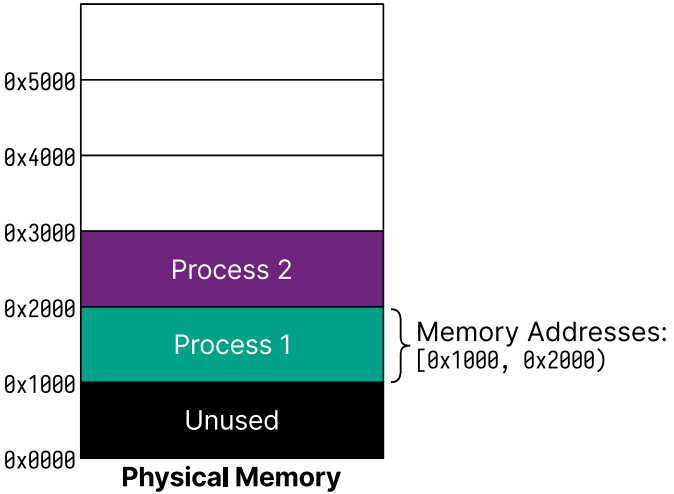
What About Global Variables?

The compiler needs to pick an address for each variable when you compile

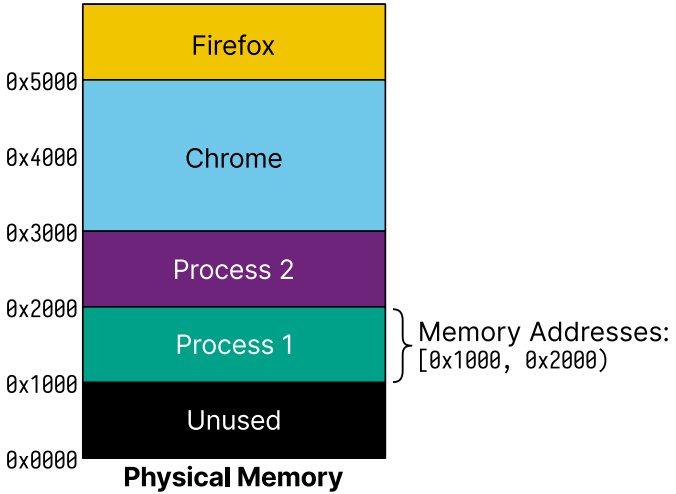
What if we had a global registry of addresses?

Would there be any issues with this?

Potential Memory Layout for Multiple Processes



Potential Memory Layout for Multiple Processes



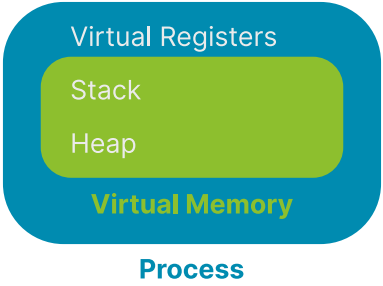
What Did We Find?

Was the address of `local` the same between the two processes?

Was the address of `global` the same between the two processes?

What else may be needed for a process?

A Process Has Its Own Virtual Memory



Acknowledgement

The slides for this course are based on materials from Professor Jon Eyolfson.